Airvo — Architecture Guide

> **Version 0.3.7** · Last updated: April 2026
>
> This document explains *how* the system works and *why* each design decision was made. It is intended for any developer who wants to understand, extend, or contribute to Airvo.

Table of Contents

1. Philosophy
2. High-Level Overview
3. Directory Structure
4. Data Flow — What Happens on Every Chat Request
5. Module Deep-Dives
   - 5.1 server.py — Application Bootstrap
   - 5.2 config/settings.py — Configuration & Persistence
   - 5.3 api/routes.py — Endpoints & Orchestration
   - 5.4 rag/ — Retrieval-Augmented Generation
   - 5.5 hardware/ — System Detection & Memory Manager
   - 5.6 discovery/ — Model Discovery
   - 5.7 cli.py — Command-Line Interface
   - 5.8 Dashboard (React)
6. Multi-Model Modes
7. TPM Guard — Rate Limit Protection
8. Security
9. API Reference
10. Configuration Files
11. Deployment & Packaging

---

1. Philosophy

Airvo follows three principles:

Principle	What it means in code
Your AI, Your Rules	No vendor lock-in. Swap models, providers, or go fully local with one click.
Zero config to start	`pip install airvo && airvo start` — ships with two free Groq models pre-configured.
Everything local	Config in `~/.airvo/`, embeddings in `~/.airvo/rag/`, no telemetry, no cloud calls unless you configure a provider.

Why FastAPI + LiteLLM?

- **FastAPI** gives us an OpenAI-compatible `/v1/chat/completions` endpoint out of the box. Any IDE client that speaks OpenAI (continue.dev, Cursor, etc.) connects with zero changes.
- **LiteLLM** is a single function `acompletion()` that routes to 100+ providers. Adding a new provider is a one-line config change, not a code change.

Why not just use LiteLLM proxy directly?

LiteLLM proxy is a generic passthrough. Airvo adds:

- Multi-model orchestration (Race, Vote, Review, Parallel)
- Dashboard with real-time config management
- RAG injection into the system prompt
- Hardware-aware memory management
- TPM guard for free-tier rate limits
- Model Discovery (Ollama catalog + OpenRouter)
- Project context memory with sanitization

---

2. High-Level Overview

```
┌──────────────────────────────────────────────────────────┐
│                continue.dev / IDE                        │
│              POST /v1/chat/completions                   │
└──────────────────────────────────────────────────────────┘
                   │
                   │  OpenAI-compatible JSON
                   ▼
┌──────────────────────────────────────────────────────────┐
│                  Airvo Server                            │
│                 (FastAPI + Uvicorn)                       │
│                                                          │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐  │
│   │ TPM Guard    │→  │ RAG Inject   │→  │ History Trim │  │
│   │ (cap tokens) │   │ (ChromaDB)   │   │ (last N msgs)│  │
│   └──────────────┘   └──────────────┘   └──────────────┘  │
│                                                  │        │
│                                                  │        │
│            ┌──────────────────────────────────┐  │        │
│            │     Mode Dispatcher              │  │        │
│            ├──────────┬──────────┬──────────┬──┤        │
│            │ Parallel │   Race   │   Vote   │  Review   │
│            └──────────┴──────────┴──────────┴──┘        │
│                 │          │          │                  │
│                 ▼          ▼          ▼                  │
│             litellm.acompletion() × N models             │
│                                                          │
└──────────────────────────────────────────────────────────┘
                   │
       ┌───────────┴────────────┐
       ▼           ▼            ▼
   ┌──────┐    ┌──────┐    ┌──────┐
   │ Groq │    │ OpenAI │    │ Ollama │   ... any provider
   └──────┘    └──────┘    └──────┘
```

Key insight: The server is an OpenAI-compatible proxy that enriches every request with context (RAG, memory, rate-limit protection) before fanning out to one or more models.

---

3. Directory Structure

```
airvo/
├── __init__.py              # Package marker
├── cli.py                   # Typer CLI — `airvo start`, `airvo config`, `airvo
version`
├── server.py                # FastAPI app, CORS, middleware, static files
├── api/
│   └── routes.py            # ALL endpoints + multi-model orchestration (807
lines)
├── config/
│   └── settings.py          # Settings singleton, file I/O, defaults (312 lines)
├── rag/
│   ├── indexer.py            # Walk dir → chunk → embed → ChromaDB (302 lines)
│   └── retriever.py          # Query index → format context for prompt (110
lines)
├── hardware/
│   ├── detector.py           # RAM, GPU, Ollama loaded models (175 lines)
│   └── memory_manager.py     # Memory pressure analysis + suggestions (145 lines)
├── discovery/
│   └── discoverer.py         # Ollama catalog + OpenRouter API (160 lines)
├── dashboard/                # Pre-built React SPA (served as static files)
│   └── dist/
└── docs/
    └── ARCHITECTURE.md       # ← You are here

dashboard/                    # React 19 + Vite source (development)
├── src/
│   ├── App.jsx               # Single-file dashboard (2943 lines, 7 languages)
│   └── main.jsx
├── vite.config.js
└── package.json

~/.airvo/                     # User config (created on first run)
├── models.json               # Model definitions + API keys
├── prefs.json                # Preferences (mode, temperature, RAG settings)
├── stats.json                # Per-model usage counters
└── rag/                      # ChromaDB vector store
    └── (chroma files)
```

Why a single `App.jsx` for the dashboard?

At 2943 lines it's large, but it keeps the dashboard **self-contained** — no complex component tree, no state management library, no build-time dependencies beyond React and Vite. The entire dashboard ships as a

single `dist/` folder inside the Python package. When the project needs more maintainability, it can be split into components without changing the architecture.

---

4. Data Flow — What Happens on Every Chat Request

Here's exactly what happens when continue.dev sends a chat request:

```
1.  POST /v1/chat/completions arrives
        │
2.  TPM GUARD — cap max_tokens for free-tier providers
    │  Scans all active models, finds the tightest _PROVIDER_LIMITS
    │  e.g. Groq → cap max_tokens from 4096 to 1500
        │
3.  SERIALIZE MESSAGES — preserve tool_calls, tool_call_id, name fields
        │
4.  BUILD SYSTEM PROMPT
    │  base: "You are an expert software development assistant."
    │  + memory: "## Project Context\n{user's notes}"  (if enabled)
    │  + RAG: "## Relevant Code\n{chunks from ChromaDB}"  (if enabled)
        │
5.  HISTORY TRUNCATION — keep system + last N non-system messages
    │  Default N=10. Prevents TPM blowup on long conversations.
        │
6.  PER-MESSAGE CHAR CAP — trim individual messages if provider has limits
    │  Only applies to Groq/Together/Cerebras/etc. Uncapped for OpenAI/Anthropic.
        │
7.  DISPATCH by mode:
    │  ├── tools present? → tool_call path (single model, streaming)
    │  ├── 1 active model → single model (streaming)
    │  └── N active models → parallel|race|vote|review (multi-model)
        │
8.  RESPONSE — SSE stream back to the IDE
```

Why this order?

- **Token guard first** (step 2): If we don't cap tokens before any branching, the LLM call can fail with a `RateLimitError` at the provider level and the user sees a cryptic error. Applying the cap unconditionally — before any mode dispatch — ensures every code path stays within the active provider's limits.
- **RAG injected into system prompt** (step 4): RAG context is added to the system message, not as a separate user message. This keeps the conversation structure clean — the model sees retrieved code as background knowledge, not as something the user said, which produces better-grounded answers.
- **History truncation before dispatch** (step 5): Running multiple models in parallel multiplies the token cost of long histories. Trimming once before dispatch keeps all paths efficient regardless of how many models are active.

---

5. Module Deep-Dives

5.1 `server.py` — Application Bootstrap

Purpose: Bootstrap the FastAPI application, set up middleware, and serve the dashboard.

92 lines. Key decisions:

Request Size Limit (10 MB)

```
MAX_REQUEST_SIZE = 10 * 1024 * 1024  # 10 MB

@app.middleware("http")
async def limit_request_size(request: Request, call_next):
    content_length = request.headers.get("content-length")
    if content_length and int(content_length) > MAX_REQUEST_SIZE:
        return JSONResponse(
            status_code=413,
            content={"detail": "Request too large. Maximum size is 10 MB."}
        )
    return await call_next(request)
```

Why 10 MB? RAG chunks, long conversation histories, and tool definitions can add up. 10 MB is generous enough for any reasonable request while protecting against accidental or malicious payloads.

CORS Configuration

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_origin_regex=r"(vscode-
webview://.*|http://localhost:\d+|http://127\.0\.0\.1:\d+)",
    allow_methods=["GET", "POST", "PATCH", "DELETE"],
    allow_headers=["Content-Type", "Authorization"],
)
```

Why `allow_origins=["*"]` AND a regex? The `*` is a fallback for non-browser clients (curl, continue.dev). The regex allows the dashboard when running in Vite dev mode (`localhost:5173`) or inside a VS Code webview panel (`vscode-webview://...`).

SPA Fallback with Path Traversal Protection

```
@app.get("/{full_path:path}")
async def serve_spa(full_path: str):
    file_path = (_dist / full_path).resolve()
    dist_resolved = _dist.resolve()
    # Block path traversal attacks
    if not str(file_path).startswith(str(dist_resolved)):
        return FileResponse(str(_dist / "index.html"))
```

```
    if file_path.exists() and file_path.is_file():
        return FileResponse(str(file_path))
    return FileResponse(str(_dist / "index.html"))
```

Why path traversal protection? Without it, a request like `GET /../../etc/passwd` could resolve outside the `dist/` folder. The `.resolve()` + `.startswith()` check ensures all paths stay within the dashboard directory.

Version from Package Metadata

```
from importlib.metadata import version as _pkg_version
_VERSION = _pkg_version("airvo")
```

Why not hardcode? The version is set once in `pyproject.toml`. Every other place (`server.py`, `cli.py`, health endpoint, dashboard) reads it from the installed package metadata. One source of truth.

5.2 `config/settings.py` — Configuration & Persistence

Purpose: All configuration, persistence to `~/.airvo/`, and the global `Settings` singleton.

312 lines. Three JSON files, one class.

File Structure

File	Purpose	Example
`~/.airvo/models.json`	Model definitions + API keys	`[{"id": "groq/llama-3.1-8b-instant", "api_key": "gsk_...", "active": true}]`
`~/.airvo/prefs.json`	User preferences	`{"mode": "parallel", "temperature": 0.7, "rag_enabled": true}`
`~/.airvo/stats.json`	Per-model usage counters	`{"groq/llama-3.1-8b-instant": {"requests": 42, "tokens": 12500}}`

Default Models (6 templates)

```
DEFAULT_MODELS = [
    # ── Pre-configured (active, free, just needs API key) ──
    {"id": "groq/llama-3.1-8b-instant",    "active": True,  "free": True},
    {"id": "groq/llama-3.3-70b-versatile",  "active": True,  "free": True},
    # ── Suggestions (inactive, need API key) ──
    {"id": "openai/gpt-4o",                 "active": False, "free": False},
    {"id": "anthropic/claude-sonnet-4-5",   "active": False, "free": False},
    {"id": "ollama/llama3",                  "active": False, "free": True},
```

```
    {"id": "lmstudio/local",                "active": False, "free": True},
]
```

Why ship 6 models but only 2 active? The first two are free Groq models — users get a working copilot immediately. The other four are templates that appear in the dashboard's "Suggestions" section, showing users what else is possible without cluttering the active model list.

Default Preferences

```
DEFAULT_PREFS = {
    "mode":             "parallel",
    "temperature":      0.7,
    "max_tokens":       1024,     # conservative default — leaves room for input
tokens, context, and RAG
    "max_history_messages": 10,
    "memory_enabled":  False,
    "memory_text":      "",
    "agent_model":      "",
    # ── RAG settings ──────────────
    "rag_enabled":      False,
    "rag_path":         "",
    "rag_max_index_mb": 200,
    "rag_max_file_kb":  500,
    "rag_top_k":        5,
    "rag_max_inject_chars": 1500,   # ~375 tokens — safe for Groq 6k TPM
}
```

Why `max_tokens: 1024`? This is the default when the IDE doesn't specify a value. A conservative output budget leaves headroom for the input side — system prompt, conversation history, and injected RAG context — without exhausting the active provider's rate limit. If the IDE sends its own `max_tokens`, the guard caps it per-provider independently.

Memory Prompt Sanitization

```
def get_memory_prompt(self) -> Optional[str]:
    text = prefs["memory_text"].strip()
    if len(text) > MEMORY_MAX_CHARS:  # 2500 chars ≈ 625 tokens
        text = text[:MEMORY_MAX_CHARS]
    # Remove null bytes and dangerous control characters
    # Keep newlines (\n), tabs (\t) and carriage returns (\r)
    text = "".join(
        ch for ch in text
        if ch in ("\n", "\t", "\r") or (ord(ch) >= 32 and ord(ch) != 127)
    )
    return text if text.strip() else None
```

Why sanitize? The memory text is user-provided and injected directly into the system prompt. Without sanitization, null bytes or control characters could confuse the tokenizer or enable prompt injection. We keep newlines and tabs (needed for code) but strip everything else.

Settings Singleton (Pydantic)

```
class Settings(BaseSettings):
    host:              str   = "localhost"
    port:              int   = 5000
    max_tokens:        int   = 4096
    temperature:       float = 0.7
    system_prompt:     str   = "You are an expert software development assistant."

    model_config = {"env_file": ".env", "env_file_encoding": "utf-8"}

    def get_models(self) -> List[dict]:      return load_models()
    def get_active_models(self) -> List[dict]: return [m for m in
self.get_models() if m.get("active")]
    def get_prefs(self) -> dict:              return load_prefs()
    def update_prefs(self, updates):          ...
    def record_usage(self, model_id, tokens): ...
    # ... CRUD methods for models, stats, prefs
```

Why Pydantic `BaseSettings`? It reads from `.env` files automatically. Users can override defaults via environment variables (`AIRVO_PORT=9000`) without touching code. The class also serves as a central access point for all config operations.

In-Memory Last-Request Tracker

```
_last_request: dict = {
    "type":  None,   # "tool_call" | "multi" | "single"
    "mode":  None,   # "parallel" | "race" | "vote" | "review"
    "model": None,   # model name (single/tool_call only)
    "count": 0,      # total requests since startup
    "tool_bypass_count": 0,
}
```

Why in-memory, not on disk? This is transient debugging info. Persisting it would add I/O on every request for data that's only useful during the current session. The `/api/health` endpoint exposes it for dashboard diagnostics.

5.3 `api/routes.py` — Endpoints & Orchestration

Purpose: Every HTTP endpoint, all Pydantic schemas, multi-model orchestration, and streaming.

807 lines. The heart of the system.

Provider Limits Table

```
_PROVIDER_LIMITS: dict[str, dict] = {
    "groq":       {"max_output": 1500, "max_msg_chars": 1500},
    "together":   {"max_output": 2000, "max_msg_chars": 2000},
    "cerebras":   {"max_output": 2000, "max_msg_chars": 2000},
    "novita":     {"max_output": 2000, "max_msg_chars": 2000},
    "openrouter": {"max_output": 2000, "max_msg_chars": 2000},
}
```

Why a lookup table? Free-tier providers enforce a strict Tokens-Per-Minute (TPM) budget that covers both input and output combined. When an IDE client sends a high `max_tokens` value (e.g. 4096), that alone can consume most of the provider's per-minute budget, leaving no room for the input side. The table lets Airvo apply conservative output caps only for providers that need them, leaving unconstrained providers (OpenAI, Anthropic, Ollama) untouched.

Why cap output to a fraction of the total budget? A typical request carrying a system prompt, RAG context, and conversation history already consumes a significant portion of the input budget. Capping output at roughly 20–25% of the provider's TPM limit ensures the input side always has room, regardless of how much context is attached to a given request.

Key Helper Functions

```
def _provider(model_id: str) -> str:
    """Extract provider prefix: 'groq/llama-3.1-8b-instant' → 'groq'"""
    return (model_id or "").split("/")[0].lower()

def _safe_max_tokens(model_id: str, requested: int) -> int:
    """Cap output tokens for providers with strict TPM limits."""
    cap = _PROVIDER_LIMITS.get(_provider(model_id), {}).get("max_output")
    return min(requested, cap) if cap else requested

def _msg_char_cap(active_models: list) -> int | None:
    """Return the tightest max_msg_chars across all active models."""
    caps = [
        _PROVIDER_LIMITS[_provider(m["id"])]["max_msg_chars"]
        for m in active_models
        if _provider(m["id"]) in _PROVIDER_LIMITS
    ]
    return min(caps) if caps else None
```

Pydantic Schemas

```
class Message(BaseModel):
    role:         str
    content:      Optional[str]  = None   # None when role=tool
```

```
    tool_calls:   Optional[List] = None   # assistant tool call requests
    tool_call_id: Optional[str]  = None   # tool result id
    name:         Optional[str]  = None   # tool name

class ChatRequest(BaseModel):
    model:        Optional[str]          = "airvo-auto"
    messages:     List[Message]
    stream:       Optional[bool]         = True
    max_tokens:   Optional[int]          = None
    temperature:  Optional[float]        = None
    tools:        Optional[List]         = None   # tool definitions from
continue.dev
    tool_choice:  Optional[str | dict]   = None

class ModelUpdate(BaseModel):
    api_key:  Optional[str]  = None
    active:   Optional[bool] = None
    base_url: Optional[str]  = None
    name:     Optional[str]  = None
    notes:    Optional[str]  = None

class NewModel(BaseModel):
    id:       str
    name:     str
    provider: str
    api_key:  Optional[str]  = None
    base_url: Optional[str]  = None
    active:   bool           = False
    free:     bool           = False
    notes:    Optional[str]  = ""

class PrefsUpdate(BaseModel):
    mode:             Optional[str]   = None
    temperature:      Optional[float] = None
    max_tokens:       Optional[int]   = None
    max_history_messages: Optional[int] = None
    memory_enabled:   Optional[bool]  = None
    memory_text:      Optional[str]   = None
    agent_model:      Optional[str]   = None
    rag_enabled:      Optional[bool]  = None
    rag_path:         Optional[str]   = None
    rag_top_k:        Optional[int]   = None
    rag_max_inject_chars: Optional[int] = None
    # ... + more RAG fields
```

Why `Optional` for most fields? The OpenAI spec makes most fields optional. continue.dev sends different combinations depending on the mode (chat vs. autocomplete vs. agent). Being permissive here means fewer client-side errors.

Tool Calling (Agent/Plan Mode)

```
if request.tools:
    agent_model_id = prefs.get("agent_model", "")
    m = next((x for x in active if x["id"] == agent_model_id), active[0])
    kwargs = dict(
        model=m["id"], messages=messages,
        max_tokens=_safe_max_tokens(m["id"], _raw_max),
        stream=True, api_key=m.get("api_key"),
        tools=request.tools,
    )
    if request.tool_choice is not None:
        kwargs["tool_choice"] = request.tool_choice
    response = await litellm.acompletion(**kwargs)
    return StreamingResponse(single_model_stream(response, m["id"]), ...)
```

Why bypass multi-model for tools? Tool calling requires a specific protocol: the model emits `tool_calls`, the client executes them, and sends back `tool` messages. Running this across multiple models would create conflicting tool call sequences. We always use a single model (configurable via `agent_model` pref) and stream the response.

`call_model()` — The Core LLM Call

```
async def call_model(model_config: dict, messages: list, request):
    prefs = settings.get_prefs()
    _capped = _safe_max_tokens(model_config["id"], _raw_max)

    kwargs = dict(
        model       = model_config["id"],
        messages    = messages,
        max_tokens  = _capped,
        temperature = request.temperature or prefs.get("temperature", 0.7),
        stream      = False,
        api_key     = model_config.get("api_key"),
    )
    if model_config.get("base_url"):
        kwargs["api_base"] = model_config["base_url"]

    response = await litellm.acompletion(**kwargs)

    # Record usage stats
    usage  = response.usage
    tokens = usage.total_tokens if usage and usage.total_tokens else 0
    settings.record_usage(model_config["id"], tokens)

    return {
        "model":   model_config["id"],
        "name":    model_config.get("name", model_config["id"]),
        "content": response.choices[0].message.content,
        "error":   None,
        "tokens":  tokens
    }
```

Why `stream=False` for multi-model? Each model runs as a non-streaming call so we can collect all results before sending the combined response. Only the final delivery to the IDE is streamed (via `multi_model_stream`).

Streaming Architecture

```
# Single model — real SSE streaming (tokens arrive as generated)
async def single_model_stream(response, model_id: str):
    async for chunk in response:
        yield f"data: {json.dumps(chunk.model_dump())}\n\n"
    yield "data: [DONE]\n\n"

# Multi-model — collect all, then fake-stream the combined text
async def multi_model_stream(results):
    combined = ""
    for r in results:
        if r["error"]:
            combined += f"**{r['name']}:**\n❌ {r['error']}\n\n---\n\n"
        else:
            combined += f"**{r['name']}:**\n{r['content']}\n\n---\n\n"

    chunk_size = 10  # characters per SSE event
    for i in range(0, len(combined), chunk_size):
        chunk = {"choices": [{"delta": {"content": combined[i:i+10]}}]}
        yield f"data: {json.dumps(chunk)}\n\n"
    yield "data: [DONE]\n\n"
```

Why fake-stream multi-model results? continue.dev expects an SSE stream for all responses. When running multiple models, we must wait for all of them to finish before we know the combined output. We then "replay" the combined text as small chunks (10 chars each), giving the IDE a smooth streaming experience.

5.4 `rag/` — Retrieval-Augmented Generation

Purpose: Index a codebase into a local vector store and inject relevant chunks into every chat request.

Architecture

```
User's codebase ──→ indexer.py ──→ ChromaDB (~/.airvo/rag/)
                                        ↑
Chat request ──→ retriever.py ──→ query embeddings ──→ top-K chunks
                                        │
                                        ↓
                          Injected into system prompt
```

`indexer.py` **(302 lines)**

Chunking strategy:

```
CHUNK_SIZE    = 400    # characters per chunk (≈ 100 tokens)
CHUNK_OVERLAP = 80     # overlap to preserve context

def _chunk_text(text: str) -> List[str]:
    chunks = []
    start = 0
    while start < len(text):
        end = start + CHUNK_SIZE
        chunks.append(text[start:end])
        start += CHUNK_SIZE - CHUNK_OVERLAP
    return [c.strip() for c in chunks if c.strip()]
```

Why 400 chars with 80 overlap? Smaller chunks (~100 tokens) retrieve more precisely. The overlap ensures that a function signature split across two chunks still appears in at least one. This is standard practice from the RAG literature.

Embedding model: `all-MiniLM-L6-v2` via sentence-transformers.

Why this model? It's only ~80 MB, runs on CPU in <1 second per batch, and produces 384-dimensional embeddings. For code retrieval, it's the best trade-off between quality and speed. Larger models (e.g., `e5-large`) are better but 10× slower.

File filtering and safety limits:

```
# Default extensions to index
extensions = [".py", ".js", ".ts", ".jsx", ".tsx", ".md", ".go", ".rs", ...]

# Default directories to skip
exclude_dirs = ["node_modules", ".git", "dist", "__pycache__", "venv", ...]

# Per-file: max 500 KB.  Total index: max 200 MB.
```

Why these limits? Indexing `node_modules` would produce millions of chunks and take hours. The 500 KB per-file limit skips generated files (minified JS, compiled binaries). The 200 MB total cap prevents runaway disk usage.

Deduplication via SHA-1:

```
def _file_hash(path: str) -> str:
    """SHA-1 of file content — used as document ID prefix."""
    h = hashlib.sha1()
    with open(path, "rb") as f:
        for block in iter(lambda: f.read(65536), b""):
            h.update(block)
    return h.hexdigest()[:16]
```

```
# Each chunk gets ID: "{file_hash}_{chunk_index}"
# ChromaDB upsert = insert-or-update, so re-indexing is idempotent
```

Why SHA-1? We only need a collision-resistant prefix for dedup, not cryptographic security. SHA-1 is fast and the 16-char hex prefix gives us 64 bits of uniqueness — more than enough for per-file chunk IDs.

`retriever.py` **(110 lines)**

```
def retrieve(query: str, top_k: int = 5) -> List[RetrievedChunk]:
    collection, embedder = _get_collection()
    query_embedding = embedder.encode([query.strip()]).tolist()
    results = collection.query(
        query_embeddings=query_embedding,
        n_results=min(top_k, collection.count()),
        include=["documents", "metadatas", "distances"],
    )
    # ChromaDB returns cosine *distance* (0 = identical, 2 = opposite).
    # Convert to similarity: score = 1 - (distance / 2)
    ...

def format_context(chunks: List[RetrievedChunk]) -> str:
    """Format as markdown for system prompt injection:

    ## Relevant Code (from your codebase)
    ### src/utils.py
    ```
    def helper(): ...
    ```
    *(relevance: 0.87)*
    """
```

Why inject into system prompt instead of user message? Putting RAG context in the system prompt keeps the conversation structure clean. The model sees it as background knowledge, not as something the user said. This produces better answers and avoids confusing multi-turn conversations.

RAG injection cap:

```
max_rag_chars = int(prefs.get("rag_max_inject_chars", 1500))
if len(ctx) > max_rag_chars:
    ctx = ctx[:max_rag_chars] + "\n... [RAG context truncated]"
```

Why cap RAG context? Injected context competes with conversation history and output tokens for the provider's rate limit budget and the model's context window. Without a hard cap, a large codebase could produce thousands of tokens of RAG context, crowding out the user's messages or triggering rate-limit errors. The cap is user-configurable so it can be raised for providers with larger context windows or higher rate limits.

5.5 `hardware/` — System Detection & Memory Manager

Purpose: Monitor RAM/GPU usage and help users manage Ollama models that consume memory.

`detector.py` **(175 lines)**

Collects three types of information:

```
@dataclass
class HardwareStatus:
    # RAM (via psutil)
    ram_total_mb: float   # e.g. 16384.0
    ram_used_mb:  float   # e.g. 12000.0
    ram_free_mb:  float   # e.g. 4384.0
    ram_percent:  float   # e.g. 73.2

    # GPU (via pynvml — NVIDIA only)
    gpus: List[GpuInfo]   # name, vram_total, vram_used, vram_free

    # Ollama (via HTTP /api/ps)
    ollama_running:       bool
    ollama_loaded_models: List[OllamaModel]  # name, size_mb, expires_at
```

Why separate libraries for each?

- **psutil** for RAM: cross-platform, well-maintained, standard choice
- **pynvml** for GPU: direct NVIDIA driver access, no CUDA dependency needed
- **urllib** for Ollama: zero external dependencies, just plain HTTP

All three are optional — the detector gracefully returns partial data if any library is missing. The endpoint always returns a response, just with `psutil_available: false` or empty GPU/Ollama arrays.

`memory_manager.py` **(145 lines)**

```
RAM_WARNING_PCT  = 75.0   # yellow warning
RAM_CRITICAL_PCT = 90.0   # red + suggest unloading

class MemoryPressure(str, Enum):
    OK       = "ok"        # RAM < 75%
    WARNING  = "warning"   # 75% ≤ RAM < 90%
    CRITICAL = "critical"  # RAM ≥ 90%

def get_suggestions(status: HardwareStatus) -> List[ModelSuggestion]:
    """
    Rules:
    - RAM > 90% + Ollama models loaded → suggest unloading largest first
    - RAM > 75% → warn about constrained resources
    - Ollama running + models loaded + RAM fine → info message
```

```
    - Everything fine → no suggestions (empty list)
    """
```

Why these thresholds? At 90% RAM, the system starts swapping to disk, making everything slow. At 75%, there's still room but the user should be aware. These are conservative values — most Ollama models need 4-8 GB of RAM.

Unloading mechanism:

```
def unload_ollama_model(model_name: str, base_url: str) -> bool:
    """Send generate request with keep_alive=0 to force immediate unload."""
    payload = json.dumps({"model": model_name, "keep_alive": 0}).encode()
    req = urllib.request.Request(url, data=payload, method="POST")
    urllib.request.urlopen(req, timeout=5)
```

Why `keep_alive=0`? This is Ollama's official API for unloading models. Setting `keep_alive` to 0 tells Ollama "unload this model immediately after the request", effectively freeing the VRAM/RAM.

5.6 `discovery/` — Model Discovery

Purpose: Help users find and add new models without leaving the dashboard.

Two discovery sources:

1. Ollama Catalog (curated, local)

```
OLLAMA_CATALOG = [
    # Tiny (< 2 GB) — fit in almost any machine
    {"id": "llama3.2:1b",      "size_gb": 0.8, "tags": ["fast", "tiny"]},
    {"id": "qwen2.5:0.5b",     "size_gb": 0.4, "tags": ["tiny", "multilingual"]},
    # Small (2–5 GB) — good for 8 GB machines
    {"id": "llama3.2:3b",      "size_gb": 2.0, "tags": ["fast", "recommended"]},
    {"id": "mistral:7b",       "size_gb": 4.1, "tags": ["balanced",
"recommended"]},
    {"id": "qwen2.5-coder:7b", "size_gb": 4.7, "tags": ["coding", "recommended"]},
    # Medium (5–12 GB) — need 16 GB machines
    {"id": "deepseek-r1:7b",   "size_gb": 4.7, "tags": ["reasoning"]},
    {"id": "phi4:14b",         "size_gb": 8.0, "tags": ["microsoft",
"reasoning"]},
    # Large (> 12 GB) — 32+ GB machines
    {"id": "llama3.1:70b",     "size_gb": 40.0, "tags": ["large"]},
    # ... 21 models total
]
```

Why curated instead of fetching Ollama's full library? Ollama has thousands of models, many of which are duplicates, quantizations, or niche variants. A curated list of 21 models covers the most useful options and lets

us show accurate size estimates and tags.

The discovery endpoint enriches each catalog entry with:

- `installed: bool` — is this model already pulled in Ollama?
- `fits_ram: bool` — does the user have enough free RAM (with 10% headroom)?

2. OpenRouter API (remote, cached)

```
_or_cache: dict = {"ts": 0.0, "data": None}
_CACHE_TTL = 300  # 5 minutes

def get_openrouter_models(limit: int = 60) -> List[dict]:
    """Fetch from https://openrouter.ai/api/v1/models, cached 5 min."""
    # Sort: free models first, then by name
    # Returns: id, name, description, context_length, is_free, prompt_cost
```

Why cache for 5 minutes? OpenRouter's model list changes infrequently. Caching avoids hammering their API and makes the dashboard feel instant on repeated visits to the Discovery page.

Quick-Add Flow

```
@router.post("/api/discovery/add")
async def discovery_add(req: QuickAddRequest):
    # Prefix model ID for litellm routing:
    # Ollama:      "ollama/{id}"      → base_url = localhost:11434
    # OpenRouter:  "openrouter/{id}"  → base_url = openrouter.ai/api/v1
    litellm_id = f"{req.provider}/{req.id}"
    new_model = {"id": litellm_id, "name": req.name, "active": False, ...}
    existing.append(new_model)
    save_models(existing)
```

Why add as inactive? Discovered models are added but not activated. The user must explicitly enable them (and provide an API key for paid models) before they affect chat responses. This prevents accidentally routing requests to unconfigured providers.

5.7 `cli.py` — Command-Line Interface

Purpose: Entry point for `airvo start`, `airvo config`, `airvo version`.

284 lines. Three commands.

Windows Encoding Fix

```
if sys.platform == "win32" and sys.stdout.encoding not in ("utf-8", "UTF-8"):
    import io
```

```
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding="utf-8",
errors="replace")
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding="utf-8",
errors="replace")
```

Why? Windows terminals default to cp1252 encoding which can't display Unicode characters (✓ ✗ ⏳ 🚀). This forces UTF-8 output with graceful fallback via `errors="replace"`.

`airvo start` **— Main Command**

```
@app.command()
def start(
    host:       str  = "127.0.0.1",
    port:       int  = 5000,
    no_browser: bool = False,
    reload:     bool = False,
):
    ensure_models_config()        # Step 1: Create ~/.airvo/models.json if needed
    ensure_continue_config(port)  # Step 2: Auto-configure continue.dev
    print_banner(host, port, ...)  # Step 3: Show startup banner
    open_browser_delayed(url)      # Step 4: Open dashboard (after 1.5s delay)
    uvicorn.run("airvo.server:app", host=host, port=port, ...)  # Step 5: Start
server
```

Auto-Configure continue.dev

```
def ensure_continue_config(port: int):
    """Write ~/.continue/config.yaml with Airvo entry if not present."""
    # If config doesn't exist → create full config
    # If config exists but no Airvo entry → append Airvo models
    # If config exists with different port → update port via regex
```

Why auto-configure? The biggest barrier to using Airvo is configuring continue.dev to point at it. By writing the config automatically, `airvo start` gives users a working copilot without any manual file editing.

continue.dev Detection

```
def detect_continue_dev() -> bool:
    """Check VS Code extensions directory for 'continue.*' folder."""
    # Windows: ~/.vscode/extensions/ + AppData/Roaming/Code/User/extensions/
    # macOS:   ~/.vscode/extensions/ + Library/Application Support/Code/...
    # Linux:   ~/.vscode/extensions/ + ~/.config/Code/User/extensions/
```

Why detect the extension? If continue.dev isn't installed, Airvo warns the user and prints the marketplace URL. This saves debugging time — "why isn't my copilot working?" is almost always "the extension isn't installed."

5.8 Dashboard (React)

Purpose: A web UI for managing models, preferences, and monitoring system health.

Tech: React 19, Vite, no external UI libraries. Single `App.jsx` file (2943 lines).

Key features:

Feature	Description
7 Languages	English, Spanish, French, German, Portuguese, Chinese, Japanese
Dark Mode	Follows system preference, toggleable
Models Page	Split into "Configured" (active or has API key) and "💡 Suggestions"
Multi-Model Modes	Switch between Parallel, Race, Vote, Review
RAG Management	Enable/disable, set path, trigger indexing, view stats
Hardware Monitor	RAM bar, GPU info, Ollama loaded models, unload button
Model Discovery	Browse Ollama catalog + OpenRouter models, quick-add
Agent Model Selector	Choose which model handles tool calls (Agent/Plan mode)
Usage Stats	Per-model request count and token usage
Configuration	Temperature, max tokens, history limit, system prompt
Help Page	Setup guide, keyboard shortcuts, FAQ

How is the dashboard served?

1. **Production:** `npm run build` → `dashboard/dist/` → served by FastAPI as static files
2. **Development:** `npm run dev` → Vite on `:5173` → API calls proxied to Airvo

```
// vite.config.js
export default defineConfig({
  server: {
    proxy: {
      '/api': 'http://127.0.0.1:8765',
      '/v1':  'http://127.0.0.1:8765',
    }
  }
})
```

Models page — Configured vs Suggestions:

The dashboard divides models into two sections:

- **Configured:** Models that are either active or have an API key set. These are models the user has intentionally set up.
- 💡 **Suggestions:** Inactive models with no API key. These are templates showing what's available. Users can configure them or remove them.

This separation keeps the active surface clean: a user with many model templates can see at a glance which models are actually doing work, without being distracted by unconfigured suggestions.

---

6. Multi-Model Modes

Airvo can run multiple models simultaneously. The mode is set via `prefs.json` or the dashboard.

Parallel (default)

```
async def parallel_mode(models, messages, request) -> list:
    """All models respond simultaneously — results shown side by side."""
    results = await asyncio.gather(*[call_model(m, messages, request) for m in
models])
    return list(results)
```

All models respond independently. Results are shown side-by-side in the IDE. **Best for:** comparing model quality, getting diverse perspectives.

Race

```
async def race_mode(models, messages, request) -> list:
    """First successful response wins, others are cancelled."""
    tasks = [asyncio.create_task(call_model(m, messages, request)) for m in
models]
    while pending:
        done, pending = await asyncio.wait(pending, return_when=FIRST_COMPLETED)
        for task in done:
            if not task.result()["error"]:
                winner = task.result()
                for t in pending: t.cancel()
                break
    winner["name"] = f"⏳ {winner['name']} (Winner)"
    return [winner]
```

All models race. First successful response wins, others are cancelled via `task.cancel()`. **Best for:** minimizing latency when you don't care which model answers.

Vote

```python
async def vote_mode(models, messages, request) -> list:
    """All respond → consensus synthesised by the first model."""
    results = await asyncio.gather(*[call_model(m, ...) for m in models])
    valid = [r for r in results if not r["error"]]

    # Build synthesis prompt with all responses
    synthesis_messages = [
        {"role": "system", "content": "You are a synthesis assistant. "
            "Identify the consensus and return the best unified answer."},
        {"role": "user", "content": f"Original question:\n{question}\n\n"
            f"Responses:\n{all_responses}\n\nSynthesise:"}
    ]
    consensus = await call_model(models[0], synthesis_messages, request)
    consensus["name"] = "📋 Consensus"
    return list(results) + [consensus]
```

All models respond, then the first model synthesises a consensus from all valid responses. **Best for:** important decisions, reducing hallucinations through majority agreement.

Review

```python
async def review_mode(models, messages, request) -> list:
    """Chain: Generator → Reviewer → (Optional) Final polish."""
    # Step 1 — first model generates  (✍️)
    current = await call_model(models[0], messages, request)

    # Steps 2..N — each subsequent model refines
    for model in models[1:]:
        refine_messages = messages + [
            {"role": "assistant", "content": current["content"]},
            {"role": "user", "content": "Review and improve this response..."}
        ]
        current = await call_model(model, refine_messages, request)

    return results  # shows each step: ✍️ → 🔍 → ✨
```

Sequential chain where each model refines the previous one's output. With 2 models: Generator → Reviewer. With 3+: Generator → Reviewer → Final Polish. **Best for:** code review, refining complex answers progressively.

---

7. Provider Rate Limit Management

Many AI providers — especially on free tiers — enforce a Tokens-Per-Minute (TPM) budget that covers both input and output tokens combined. IDE clients like continue.dev typically request large output windows (`max_tokens=4096`) without knowledge of the active provider's limits. Without intervention, this silently exhausts the budget before the input side (system prompt, history, RAG context) is even counted, resulting in a `RateLimitError` that the user sees as a mysterious failure.

Airvo addresses this with **three defensive layers** that work together:

The Three Layers

Layer 1 — Early cap at handler entry (lines ~296-313 of routes.py):

```
@router.post("/v1/chat/completions")
async def chat_completions(request: ChatRequest):
    # Runs BEFORE any code path (tool_call, single, multi)
    _active_providers = {_provider(m["id"]) for m in settings.get_active_models()}
    _lowest_cap = None
    for p in _active_providers:
        c = _PROVIDER_LIMITS.get(p, {}).get("max_output")
        if c and (_lowest_cap is None or c < _lowest_cap):
            _lowest_cap = c
    if _lowest_cap and request.max_tokens and request.max_tokens > _lowest_cap:
        logger.warning("[TPM-GUARD] request.max_tokens %d → capped to %d",
                       request.max_tokens, _lowest_cap)
        request.max_tokens = _lowest_cap  # 4096 → 1500
```

Layer 2 — Per-call cap in `call_model()`:

```
kwargs = dict(
    max_tokens=_safe_max_tokens(model_config["id"], _raw_max),
    ...
)
```

Layer 3 — Per-message content trim:

```
_char_cap = _msg_char_cap(settings.get_active_models())
if _char_cap:
    for m in other_msgs:
        if isinstance(m.get("content"), str) and len(m["content"]) > _char_cap:
            m["content"] = m["content"][:_char_cap] + " … [trimmed]"
```

Why three layers? Defense in depth. Layer 1 catches the most common case (IDE sending high `max_tokens`). Layer 2 handles per-model differences when running multi-model. Layer 3 prevents long individual messages from blowing the input budget. If any single layer fails, the others still protect.

---

8. Security

Measure	Where	Why
Path traversal protection	`server.py` SPA fallback	Prevents `../../etc/passwd` via `.resolve()` + `.startswith()`

Measure	Where	Why
Request size limit	`server.py` middleware	10 MB cap prevents denial-of-service via oversized payloads
Memory sanitization	`settings.py` `get_memory_prompt()`	Strips null bytes and control chars from user-provided text
CORS regex	`server.py`	Only allows `localhost`, `127.0.0.1`, and `vscode-webview://`
Bind to 127.0.0.1	`cli.py` default host	Server not accessible from the network by default
API keys in user dir	`~/.airvo/models.json`	Keys stored locally, never sent except to the configured provider
No telemetry	Everywhere	No analytics, no tracking, no phone-home. Fully offline capable.
History truncation	`routes.py`	Limits conversation length to prevent token-based attacks
RAG content cap	`routes.py`	Hard limit on injected RAG content prevents prompt flooding

9. API Reference

Base URL: `http://127.0.0.1:8765` (default dev port) or `http://localhost:5000` (default `airvo start`).

OpenAI-Compatible Endpoints

Method	Path	Description
`POST`	`/v1/chat/completions`	Chat completion with SSE streaming. Supports `tools` and `tool_choice` for Agent/Plan mode.
`GET`	`/v1/models`	List available models. Returns `{"data": [{"id": "airvo-auto"}]}`.

Models CRUD

Method	Path	Description
`GET`	`/api/models`	List all configured models (active + inactive).
`GET`	`/api/models/active`	List only active models.
`POST`	`/api/models`	Add a new model. Body: `NewModel`.
`PATCH`	`/api/models/{id}/toggle`	Toggle active/inactive. Query param: `active=true|false`.
`PATCH`	`/api/models/{id}/key`	Set API key. Query param: `api_key=sk-...`.
`PATCH`	`/api/models/{id}`	Update model fields. Body: `ModelUpdate`.

Method	Path	Description
DELETE	/api/models/{id}	Delete a model permanently.

Preferences

Method	Path	Description
GET	/api/prefs	Get all current preferences (mode, temperature, RAG settings, etc.).
PATCH	/api/prefs	Update one or more preferences. Body: PrefsUpdate (partial).

Stats & Health

Method	Path	Description
GET	/api/stats	Per-model usage stats: {model_id: {requests, tokens}}.
DELETE	/api/stats	Reset all usage statistics to zero.
GET	/api/health	Server status, version, active models, last request info.

RAG (Retrieval-Augmented Generation)

Method	Path	Description
GET	/api/rag/status	Index stats: files indexed, chunk count, size, RAG availability.
POST	/api/rag/index	Trigger (re)indexing. Body: optional RagIndexRequest overrides.
DELETE	/api/rag/reset	Wipe the entire RAG index (all embeddings and metadata).

Hardware & Memory

Method	Path	Description
GET	/api/hardware/status	RAM, GPU, Ollama status, memory pressure, unload suggestions.
POST	/api/hardware/unload	Unload an Ollama model. Body: {"model_name": "llama3"}.

Model Discovery

Method	Path	Description
GET	/api/discovery/ollama	Curated Ollama catalog with installed and fits_ram flags.
GET	/api/discovery/openrouter	OpenRouter models (cached 5 min). Query: limit=60.
POST	/api/discovery/add	Quick-add a discovered model to Airvo settings. Body: QuickAddRequest.

Interactive API Documentation

FastAPI auto-generates interactive docs at:

- **Swagger UI:** `/docs` — try endpoints live, see schemas
- **ReDoc:** `/redoc` — clean read-only reference

---

10. Configuration Files

`~/.airvo/models.json`

```
[
  {
    "id": "groq/llama-3.1-8b-instant",
    "name": "Llama 3.1 8B (Groq)",
    "provider": "groq",
    "api_key": "gsk_abc123...",
    "base_url": null,
    "active": true,
    "free": true,
    "notes": "Fast and free — great for quick tasks"
  },
  {
    "id": "openai/gpt-4o",
    "name": "GPT-4o",
    "provider": "openai",
    "api_key": null,
    "base_url": null,
    "active": false,
    "free": false,
    "notes": "Requires OpenAI API key — platform.openai.com"
  }
]
```

`~/.airvo/prefs.json`

```
{
  "mode": "parallel",
  "temperature": 0.7,
  "max_tokens": 1024,
  "max_history_messages": 10,
  "memory_enabled": true,
  "memory_text": "This is a Python/React project using FastAPI...",
  "agent_model": "groq/llama-3.3-70b-versatile",
  "rag_enabled": true,
  "rag_path": "/home/user/my-project",
  "rag_top_k": 5,
  "rag_max_inject_chars": 1500,
  "rag_max_index_mb": 200,
  "rag_max_file_kb": 500,
  "rag_extensions": [".py", ".js", ".ts", ".md"],
```

```
  "rag_exclude_dirs": ["node_modules", ".git", "dist"]
}
```

`~/.airvo/stats.json`

```
{
  "groq/llama-3.1-8b-instant": {
    "requests": 142,
    "tokens": 45200
  },
  "groq/llama-3.3-70b-versatile": {
    "requests": 87,
    "tokens": 62100
  }
}
```

`~/.continue/config.yaml` (auto-generated by `airvo start`)

```
name: Local Config
version: 1.0.0
schema: v1
models:
  - name: Airvo
    provider: openai
    model: airvo-auto
    apiBase: http://localhost:5000/v1
    apiKey: local
    roles:
      - chat
      - edit
      - apply
  - name: Airvo Autocomplete
    provider: openai
    model: airvo-auto
    apiBase: http://localhost:5000/v1
    apiKey: local
    roles:
      - autocomplete
```

---

11. Deployment & Packaging

Install from PyPI

```
pip install airvo           # Core (Groq, OpenAI, Anthropic, Ollama, etc.)
pip install airvo[rag]      # + RAG dependencies (chromadb, sentence-transformers)
```

```
pip install airvo[hardware] # + Hardware monitoring (psutil, pynvml)
pip install airvo[all]      # Everything
```

Build the Dashboard

```
cd dashboard
npm install
npm run build
# Output → dashboard/dist/ (copied to airvo/dashboard/dist/ during packaging)
```

Start the Server

```
# Production
airvo start --port 8765

# Development (with hot reload)
airvo start --port 8765 --reload

# Don't open browser
airvo start --no-browser

# Background (Windows, no console window)
python _start_server.py
```

Other CLI Commands

```
# Show current version
airvo version

# View config
airvo config --show

# Set Telegram token (for future integration)
airvo config --telegram-token "BOT_TOKEN"
```

Publish to PyPI

```
# 1. Bump version in pyproject.toml
# 2. Build
python -m build
# 3. Upload
twine upload dist/*
```

Package Structure (pyproject.toml)

```
[project]
name = "airvo"
version = "0.3.7"
dependencies = [
    "fastapi>=0.100",
    "uvicorn",
    "litellm>=1.0",
    "typer",
    "python-dotenv",
    "pydantic-settings",
]

[project.optional-dependencies]
rag = ["chromadb>=0.4", "sentence-transformers>=2.0"]
hardware = ["psutil>=5.9", "pynvml>=11.0"]
all = ["airvo[rag]", "airvo[hardware]"]

[project.scripts]
airvo = "airvo.cli:app"
```

Metrics (v0.3.7)

Component	Lines of Code
`api/routes.py`	807
`config/settings.py`	312
`rag/indexer.py`	302
`cli.py`	284
`hardware/detector.py`	175
`discovery/discoverer.py`	160
`hardware/memory_manager.py`	145
`rag/retriever.py`	110
`server.py`	92
Python total	**~2,387**
`dashboard/src/App.jsx`	2,943
Full stack total	**~5,330**

Feature	Count
API endpoints	22

Feature	Count
Pydantic schemas	7
Multi-model modes	4
Dashboard languages	7
Default model templates	6
Ollama catalog entries	21
Supported file extensions (RAG)	24
Provider rate-limit configs	5

End of architecture guide. For questions or contributions, open an issue on GitHub.